

IAW · UNIDAD 4

PHP: la cocina de la web

Cómo Nginx consigue que una página se «cocine» en el momento

HOY

Tres cosas, nada más

1 

Entender cómo funciona una web con PHP

Con un restaurante

2 

Conectar Nginx con PHP

Cinco líneas de configuración

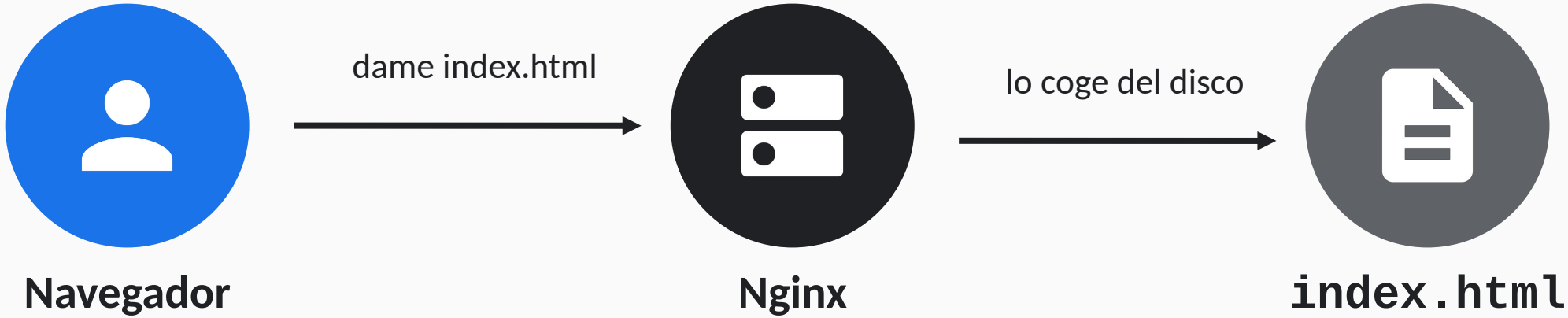
3 

Arreglar un fallo de seguridad

Un formulario que se deja engañar

HASTA AHORA

Web estática: el servidor entrega algo que ya está hecho



Como un menú impreso: todo el mundo ve lo mismo, y para cambiarlo hay que reimprimirlo.

ESTÁTICA — El servidor ENTREGA un fichero. No piensa, no calcula, no cambia nada.

LO NUEVO

Web dinámica: la página se fabrica en el momento

Moodle

Tus notas, tus cursos: tu página es distinta de la de tu compañero

La web del banco

Tu saldo de hoy, a esta hora

Una tienda

Tu carrito, tus pedidos

Un formulario

Escribes algo y el servidor lo guarda

Esas páginas no están en el disco: un PROGRAMA las fabrica en cada visita.

ESE PROGRAMA — Hoy: PHP. Mueve WordPress, Moodle y Nextcloud, que veréis este curso.

LA IDEA DE HOY

Una web con PHP es un restaurante



El camarero NO cocina. Nginx no sabe ejecutar PHP: se lo pasa a la cocina, que se llama PHP-FPM.

RECUERDA — Cliente = navegador · Camarero = Nginx · Cocina = PHP-FPM

RECETA Y PLATO

El .php es la receta; el HTML es el plato

index.php · LA RECETA

```php
<p>Son las <?= date('H:i') ?></p>

<?php foreach ($libros as $l): ?>
  <li><?= $l ?></li>
<?php endforeach; ?>
```

instrucciones

lo que recibe el navegador · EL PLATO

```html
<p>Son las 18:05</p>

  <li>El camino</li>
  <li>Nada</li>
  <li>Luces de bohemia</li>
```

el resultado

Al cliente se le sirve el plato. La receta se queda en la cocina.

LA COCINA — PHP lee la receta, la ejecuta y devuelve HTML. El navegador nunca ve el PHP.

EL PELIGRO

¿Y si el camarero te da la receta en vez del plato?



Camarero

Nginx mal configurado

la receta

Cliente

se descarga el .php

Se descarga el código entero. Y dentro puede estar la contraseña de la base de datos.

POR ESO — Hay que decirle al camarero, con una regla, que los .php van a la cocina.

LA REGLA PARA EL CAMARERO

Cinco líneas en el fichero de Nginx

nginx/sites-available/club.conf

```
location ~ \.php$ {
  try_files $uri =404;
  fastcgi_pass  php:9000;
  include       fastcgi_params;
  fastcgi_param SCRIPT_FILENAME
    $document_root$fastcgi_script_name;
}
```

Si acaba en .php…
`location ~ \.php$`

…y la receta existe…
`try_files`

…llévalo a la cocina…
`fastcgi_pass php:9000`

…y dile qué receta es.
`SCRIPT_FILENAME`

En castellano: «lo que acabe en .php, a la cocina».

NO SE MEMORIZA — Se entiende. La regla está en los apuntes y en el kit.

DEMO · PASO 1

Sin la regla: el camarero te da la receta

en el cliente: docker compose exec cliente sh

```
$ curl -kI https://club.iaw.local/
Content-Type: application/octet-stream        ← «un fichero cualquiera»

$ curl -k https://club.iaw.local/
<?php                                        ← ¡la receta!
$titulo = 'Inicio · Club de Lectura';
```

En el navegador: https://localhost:8445

El fichero index.php se DESCARGA. Ábrelo: es el código.

FÍJATE — Content-Type: application/octet-stream = «te doy un fichero, haz lo que quieras con él».

DEMO · PASO 2

Ponemos la regla: tres pasos de siempre

1

Edita

Quita las # del bloque location ~ \.php$

`club.conf`

2

Comprueba

¿Está bien escrito?

`nginx -t`

3

Recarga

Nginx lee la regla nueva sin cortar el servicio

`nginx -s reload`

terminal

```
$ docker compose exec nginx nginx -t   &&   docker compose exec nginx nginx -s reload
```

SI nginx -t FALLA — No recargues. Lee el error: dice la línea que está mal.

DEMO · PASO 3

Con la regla: el camarero te trae el plato

en el cliente: docker compose exec cliente sh

```
$ curl -kI https://club.iaw.local/
Content-Type: text/html; charset=UTF-8        ← ¡una página!
X-Powered-By: PHP/8.3.x                       ← (lo quitamos el lunes)

$ curl -k https://club.iaw.local/
<h2>Lecturas del trimestre (4)</h2>          ← el plato, ya cocinado
```

En el navegador

Sale la página con la tabla de libros. El código no se ve por ningún lado.

LO CONSEGUIDO — El camarero ya sabe que los .php van a la cocina.

CUANDO ALGO FALLA

Tres errores, tres cosas que pasan en un restaurante

404



«¿Qué receta?»

El camarero no dice qué plato.
Falta SCRIPT_FILENAME.

502



Cocina cerrada

Nadie contesta.
Puerto mal o PHP apagado.

504



El plato tarda demasiado

La cocina contesta, pero tarde.
Una página muy lenta.

502 FRENTE A 504 — 502: no abre nadie. 504: abren, pero tardan demasiado.

SI FALLA

Dónde mirar, siempre en este orden

1

¿Qué código da?

```
docker compose exec cliente curl -kI https://club.iaw.local/
```

2

¿Qué dice el camarero?

```
logs/club_error.log
```

3

¿Qué dice la cocina?

```
docker compose logs php
```

SIEMPRE — Primero el código, después el log de Nginx y por último el de PHP.

LEER UNA RECETA

PHP: solo cuatro piezas

`<?php … ?>`	Aquí empieza y acaba el PHP. Lo de fuera es HTML normal
`$libros`	Una variable: una caja con un nombre. Siempre con $
`<?= $titulo ?>`	Imprime: pon aquí lo que hay en la caja
`foreach`	Repite: una vez por cada libro

PRUÉBALO — Cambia un libro en www/club/inc/libros.php y recarga la página: cambia al momento.

Lo que escribe el usuario llega a PHP

navegador

```
https://localhost:8445/recomendar.php?nombre=Ana&libro=Nada
```



recomendar.php

```php
$nombre = $_GET['nombre'];        // vale "Ana"
<p>Gracias, <?= $nombre ?>.</p>
```

Lo que va detrás de ? en la URL lo escribe el usuario. Puede escribir CUALQUIER COSA.

NUNCA — Contraseñas en la URL: se quedan en el historial y en los logs.

EL FALLO DE SEGURIDAD

XSS: el cliente escribe código… y la página lo ejecuta

navegador

```
recomendar.php?nombre=<script>alert('hola')</script>
```

¡Salta una alerta! La página ha ejecutado lo que escribió el usuario. Con otro código, podría robar tu sesión.

Es como si la cocina obedeciera lo que el cliente escribe en la comanda.

SE LLAMA — XSS (Cross-Site Scripting). Es uno de los fallos más comunes de la web.

EL ARREGLO

Una función: htmlspecialchars

ANTES ✘

```
<p>Gracias, <?= $nombre ?>.</p>
```

DESPUÉS ✔

```
<p>Gracias,
<?= htmlspecialchars($nombre) ?>
.</p>
```

Salta la alerta

El navegador obedece el <script>

Se ve como texto

Sale <script>… escrito, sin hacer nada

Lo que escribió el usuario, antes de imprimirlo: htmlspecialchars.

REGLA DE ORO — Nunca te fíes de lo que escribe el usuario.

TU TURNO

Cinco pasos, en parejas

1	Arranca el laboratorio	`docker compose up -d --build`
2	Quita las # de la regla	`nginx/sites-available/club.conf`
3	Comprueba y recarga	`nginx -t` y `nginx -s reload`
4	Arregla el formulario	`htmlspecialchars en www/club/recomendar.php`
5	Comprueba	`bash comprobar.sh` → 4 de 10

SI ACABAS ANTES — Borra la línea SCRIPT_FILENAME, recarga: ¿qué error sale? ¿Por qué?

LO QUE NOS LLEVAMOS HOY

Resumen de puntos clave

- Navegador = cliente · Nginx = camarero · PHP-FPM = cocina
- El .php es la receta; el HTML, el plato
- Sin la regla location ~ \.php$, se descarga el código
- 404 «¿qué receta?» · 502 cerrado · 504 tarda
- Lo que escribe el usuario: htmlspecialchars

EL LUNES

PHP listo para un servidor de verdad

- Que no enseñe sus errores
- Que no diga su versión
- ¿Cuántos cocineros necesita?

El camarero no cocina: lleva los .php a la cocina y sirve el plato.



IAW · UNIDAD 4 · SESIÓN 2

PHP listo para un servidor de verdad

1. Que la cocina no enseñe sus errores ni su versión
2. ¿Cuántos cocineros necesita?
3. Encontrar la receta que tarda

PHP.INI

Las normas de la cocina: cuatro líneas

php/zz-lab.ini

```
expose_php          = Off
display_errors      = Off
log_errors          = On
error_log           = /var/log/lab/php-errores.log
disable_functions   = exec,shell_exec,system,…
upload_max_filesize = 4M
```

No dice su versión
Sin la cabecera X-Powered-By

No enseña sus errores
Van a un log, no al cliente

Sin cuchillos peligrosos
Un .php malicioso no ejecuta órdenes

Pedidos con tamaño máximo
Subidas de 4 MB como mucho

aplicar y comprobar

```
$ docker compose restart php
```

```
$ docker compose exec cliente curl -kI https://club.iaw.local/
```

SI NO CAMBIA NADA — PHP-FPM lee el php.ini al arrancar: sin restart, sigue con las normas viejas.

ERRORES

Si se quema algo, el cocinero no sale a contarlo

navegador · display_errors = On ✘

```
Warning: require(/var/www/club/inc/
  configuracion-que-no-existe.php):
  Failed to open stream …
Fatal error: Uncaught Error: Failed
  opening required … in
  /var/www/club/error.php:4
```

el cliente ve tus rutas

logs/php-errores.log · display_errors = Off ✔

```
PHP Fatal error:  Uncaught Error:
  Failed opening required
  '/var/www/club/inc/…'
  in /var/www/club/error.php:4

(el usuario solo ve un 500)
```

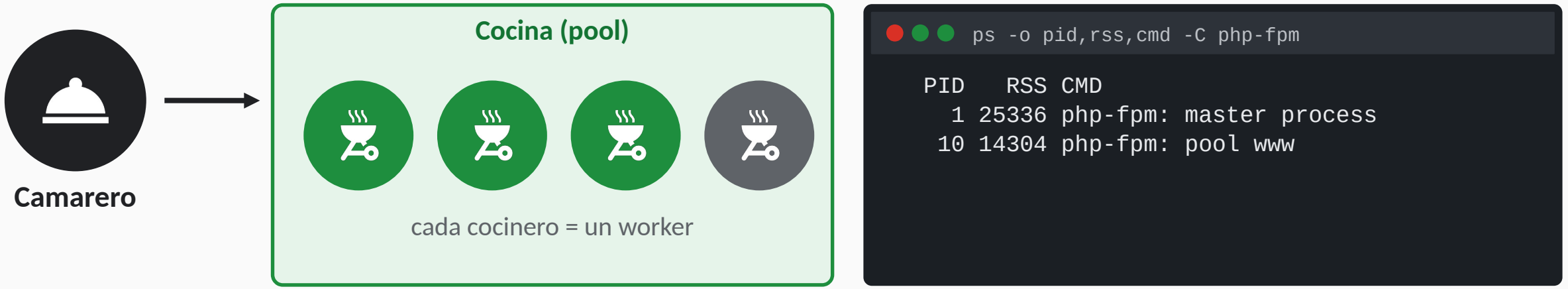
solo tú lo lees

En producción: display_errors = Off y log_errors = On. Siempre.

EN DESARROLLO — En tu ordenador sí puedes ver los errores (display_errors = On). En el servidor, nunca.

EL POOL

PHP-FPM tiene un equipo de cocineros




```
ps -o pid,rss,cmd -C php-fpm
  PID   RSS CMD
    1 25336 php-fpm: master process
   10 14304 php-fpm: pool www
```

pm	Cuántos cocineros	Cuándo
`dynamic`	Entre un mínimo y un máximo	Lo normal
`static`	Siempre max_children	Mucho tráfico constante
`ondemand`	Los llama al llegar pedidos	Poco tráfico, poca memoria

EL NÚMERO MÁXIMO — pm.max_children: cuántos cocineros caben en la cocina a la vez.

EL CÁLCULO

¿Cuántos cocineros caben en la cocina?

Memoria de la cocina

512 MB

mem_limit del compose

Lo que usa el resto

− 112 MB

sistema, master…

Lo que ocupa un cocinero

÷ 40 MB

con la aplicación real

= 10

cocineros

php/zz-lab-pool.conf

```
pm = dynamic      pm.max_children = 10      ; (512 − 112) ÷ 40 = 10
```

SIN MEDIR NO HAY CÁLCULO — El 5 de fábrica no es una respuesta. Tu número, con tu cuenta, sí.

SATURACIÓN

Más pedidos que cocineros: cola en la puerta

2 cocineros

6 pedidos a la vez → 8,0 segundos

10 cocineros

Los mismos 6 pedidos → 4,1 segundos

```
docker compose logs php
```

```
WARNING: [pool www] server reached pm.max_children setting (2), consider raising it
```

Si la cola dura demasiado → 504, aunque el servidor esté tranquilo.

FÍJATE — Un 504 no siempre es «el servidor va lento»: a veces faltan cocineros.

RENDIMIENTO

Quién tarda (slowlog) y recetas de memoria (OPcache)

slowlog

¿En qué línea se atasca el cocinero?

logs · php-lento.log

```
script_filename = /var/www/club/lento.php
[0x…] sleep() /var/www/club/lento.php:5
```

OPcache

La receta, aprendida de memoria

el pool y el php.ini

```
request_slowlog_timeout = 3s    ; pool
opcache.enable = 1              ; php.ini
opcache.validate_timestamps = 1 ; php.ini
```

slowlog dice la línea exacta. OPcache evita releer la receta.

TRAMPA — validate_timestamps = 0 y cambias un .php… y sigue saliendo lo viejo: reinicia PHP-FPM.

TU TURNO

Del 4 al 10 de 10, en parejas

1	**php.ini**	php/zz-lab.ini: las cuatro líneas + restart php
2	**Errores**	error.php: ¿qué ve el cliente? ¿Qué hay en el log?
3	**Pool**	Mide el RSS, calcula pm.max_children, escribe tu cuenta
4	**Cierra**	info.php e inc/ fuera: deny all en club.conf
5	**Comprueba**	bash comprobar.sh → 10 de 10

SI ACABAS ANTES — Pon pm.max_children = 2 y lanza el ab de la demo: ¿cuánto tarda? ¿Qué dice el log?

LO QUE NOS LLEVAMOS DE LA UD4

Resumen de puntos clave

- expose_php = Off: la cocina no dice su versión
- display_errors = Off: los errores, al log
- El pool = los cocineros; cada .php ocupa uno
- pm.max_children = memoria libre ÷ memoria de un cocinero
- slowlog: la línea que tarda · OPcache: receta de memoria

LA UD5

La misma idea, con Python

- La cocina: Gunicorn
- El camarero: Nginx con proxy_pass
- Una API que devuelve JSON

Una cocina segura no enseña nada al cliente y tiene los cocineros que caben.